

Laporan Tugas Praktikum4



Syahrul Giga Wahyudi - 0110224085

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

0110224085@student.nurulfikri.ac.id

Logistic Regression

Praktikum ini bertujuan untuk membangun dan mengevaluasi model **Regresi Logistik** sebagai solusi untuk masalah klasifikasi biner, yaitu memprediksi status **stunting** pada balita. Langkah kunci mencakup persiapan data (**preprocessing, scaling, encoding**), pembentukan model, dan analisis performa model menggunakan berbagai metrik evaluasi standar (**Akurasi, Presisi, Recall, F1-Score, dan ROC-AUC**)

1.1 Load data dari Google drive dan import library



```
[1] Import Library
# 10

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report, RocCurveDisplay, ConfusionMatrixDisplay
)

[2] from google.colab import drive
# 20
drive.mount('/content/drive')

Mounted at /content/drive
```

Gambar 1.1 kode dan output

Penjelasan

Langkah awal adalah mengimpor *library* utama. **Pandas** dan **NumPy** digunakan untuk pemrosesan dan manajemen data. Dari *scikit-learn* (*sklearn*), beberapa modul penting diimpor: **train_test_split** untuk membagi data; **StandardScaler** untuk menyamakan skala fitur; **LogisticRegression** sebagai model klasifikasi; dan **Pipeline** untuk mengintegrasikan tahapan *preprocessing* dan pemodelan secara terstruktur

Bagian kode `from google.colab import drive` ini digunakan untuk **menghubungkan Google Colab dengan akun Google Drive mu..** Setelah kode ini dijalankan, akan muncul tautan otorisasi. Kamu harus mengklik tautan tersebut, memilih akun Google, dan memberikan izin agar Colab bisa mengakses file-file yang tersimpan di Google Drive-mu. Proses ini hanya perlu dilakukan satu kali per sesi.

1.2 membaca file csv

✓ Membaca data file CSV

Klik dua kali (atau tekan Enter) untuk mengedit

3]
2 d



```
df = pd.read_csv("/content/drive/MyDrive/Pratikum_M1/pratikum04/data/stunting_wasting_dataset.csv")
df.head()
```



	Jenis Kelamin	Umur (bulan)	Tinggi Badan (cm)	Berat Badan (kg)	Stunting	Wasting
0	Laki-laki	19	91.6	13.3	Tall	Risk of Overweight
1	Laki-laki	20	77.7	8.5	Stunted	Underweight
2	Laki-laki	10	79.0	10.3	Normal	Risk of Overweight
3	Perempuan	2	50.3	8.3	Severely Stunted	Risk of Overweight
4	Perempuan	5	56.4	10.9	Severely Stunted	Risk of Overweight



Langkah berikutnya:

[Buat kode dengan df](#)

[New interactive sheet](#)

Gambar 1.2 kode dan output

Penjelasan

Data diambil menggunakan pustaka pandas dengan fungsi `read_csv()`. Dataset berisi informasi mengenai jenis kelamin, umur, tinggi, dan berat badan balita. Setelah data dimuat ke dalam DataFrame, proses analisis dapat dilakukan secara lebih mudah dan terstruktur.

1.3 informasi data set

✓ Melihat informasi umum dataset

```
[4]
✓ 0 d #melihat informasi umum data set
df.info ()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Jenis Kelamin         100000 non-null object  
1   Umur (bulan)          100000 non-null int64   
2   Tinggi Badan (cm)     100000 non-null float64  
3   Berat Badan (kg)      100000 non-null float64  
4   Stunting              100000 non-null object  
5   Wasting               100000 non-null object  
dtypes: float64(2), int64(1), object(3)
memory usage: 4.6+ MB
```

Gambar 1.3 kode dan output

Penjelasan

Metode `df.info()` digunakan untuk melihat gambaran umum struktur data. Fungsi ini menampilkan jumlah baris dan kolom, nama kolom, jumlah data yang terisi, serta tipe data pada setiap kolom, seperti `int64` untuk angka dan `object` untuk teks.

1.4 Cek Missing Value

✓ Cek Missing Value

```
[5]
✓ 0 d #cek missing value
df.isnull().sum()

Jenis Kelamin    0
Umur (bulan)     0
Tinggi Badan (cm) 0
Berat Badan (kg) 0
Stunting         0
Wasting          0

dtype: int64
```

Gambar 1.4 kode dan output

Penjelasan

Perintah ini digunakan untuk memastikan apakah terdapat data yang hilang pada setiap kolom. Hasil menunjukkan seluruh kolom (Jenis Kelamin, Umur, Tinggi, Berat, Stunting, Wasting) memiliki nilai 0 pada jumlah missing value. Artinya, tidak ada data kosong, sehingga dataset dapat langsung digunakan tanpa proses imputasi.

1.5 Cek Nilai Unik

✓ Cek Nilai Unik

```
[6]
✓ 0 d #cek nilai uniq
      df['Stunting'].unique()

array(['Tall', 'Stunted', 'Normal', 'Severely Stunted'], dtype=object)

[7]
✓ 0 d df['Jenis Kelamin'].unique()

array(['Laki-laki', 'Perempuan'], dtype=object)
```

Gambar 1.5 kode dan output

Penjelasan

1. Kolom Stunting memiliki 4 kategori: Tall, Stunted, Normal, Severely Stunted.
2. Kolom Jenis Kelamin memiliki 2 kategori: Laki-laki dan Perempuan.
3. Karena target prediksi (stunting) bersifat biner (ya/tidak), maka kategori ini akan disederhanakan menjadi dua kelas: • 1 → Stunted / Severely Stunted • 0 → Normal / Tall 1.6

1.6 Mapping Kolom Kategorik ke Bentuk Numerik

✓ Mapping Kolom Kategorik ke Bentuk Numerik

```
[8]
✓ 0 d # 1. Mapping kolom Stunting -> biner
      map_stunt = {'Stunted': 1, 'Severely Stunted': 1, 'Normal': 0, 'Tall': 0}
      df['Stunting_bin'] = df['Stunting'].map(map_stunt).astype('Int64')

      # 2. Mapping kolom Jenis Kelamin -> biner (Laki-laki = 1, Perempuan = 0)
      df['JK_bin'] = (df['Jenis Kelamin'] == 'Laki-laki').astype(int)

      print("Distribusi Stunting_bin: \n", df['Stunting_bin'].value_counts())
      print("\nDistribusi JK_bin: \n", df['JK_bin'].value_counts())

Distribusi Stunting_bin:
Stunting_bin
0      78021
1      21979
Name: count, dtype: Int64

Distribusi JK_bin:
JK_bin
1      50179
0      49821
Name: count, dtype: int64
```

Gambar 1.6 kode dan output

Penjelasan

- Kode di atas mengubah nilai teks menjadi bentuk numerik agar dapat diproses model.
- Kolom baru Stunting_bin digunakan sebagai target (Y).
- Kolom JK_bin digunakan sebagai fitur (X) tambahan. •

Hasil distribusi: o Stunting_bin → 78.021 data tidak stunting (0) dan 21.979 data stunting (1). o JK_bin → 50.179 laki-laki (1) dan 49.821 perempuan (0).

- Distribusi ini menunjukkan data relatif seimbang antara laki-laki dan perempuan, namun kelas “stunting” lebih sedikit dibanding “tidak stunting”.

1.7 Analisis Korelasi Antar Variabel Numerik

Analisis Korelasi Antar Variabel Numerik

+ Kode

+ Teks

```
#Analisis Korelasi Antar Variabel Numerik
corr_matrix = df.corr(numeric_only=True)
corr_matrix
```

	Umur (bulan)	Tinggi Badan (cm)	Berat Badan (kg)	Stunting_bin	JK_bin
Umur (bulan)	1.000000	0.875869	0.665389	0.038630	0.004046
Tinggi Badan (cm)	0.875869	1.000000	0.626005	-0.283855	0.073505
Berat Badan (kg)	0.665389	0.626005	1.000000	0.021090	0.045797
Stunting_bin	0.038630	-0.283855	0.021090	1.000000	-0.005981
JK_bin	0.004046	0.073505	0.045797	-0.005981	1.000000

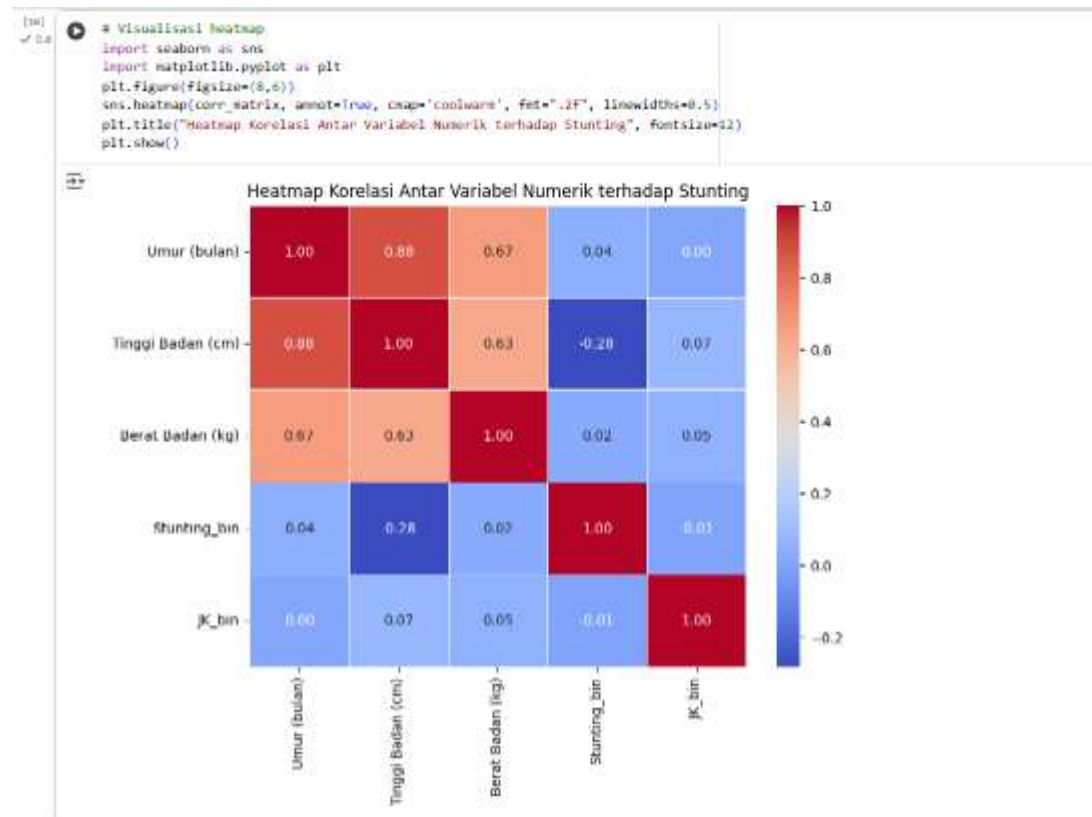
Langkah berikutnya: [Buat kode dengan corr_matrix](#) [New interactive sheet](#)

Gambar 1.7 kode dan output

Penjelasan

- Nilai mendekati 1 → hubungan positif kuat (naik bersama)
- Nilai mendekati -1 → hubungan negatif kuat (berlawanan arah)
- Nilai mendekati 0 → hubungan lemah atau tidak ada hubungan

1.8 Visualisasi Heatmap Korelasi



Gambar 1.8 kode dan output

Penjelasan

Visualisasi heatmap korelasi digunakan untuk menganalisis hubungan antar variabel numerik secara keseluruhan dalam dataset. Melalui heatmap ini, dapat diketahui variabel-variabel yang memiliki tingkat korelasi kuat, sedang, atau lemah, serta arah hubungan positif maupun negatif terhadap variabel target `Stunting_bin`.

2.1 Menentukan Fitur dan Target

Menentukan Fitur dan Target

```
[11]
✓ 0 d # Fitur numerik dan gender
feature_num = ['Umur (bulan)', 'Tinggi Badan (cm)', 'Berat Badan (kg)']
feature_bin = ['JK_bin']

# Gabungkan & drop missing (meskipun pada cek awal tidak ada missing value)
use_cols = feature_num + feature_bin + ['stunting_bin']
df_model = df[use_cols].dropna().copy()

X = df_model[feature_num + feature_bin]
y = df_model['stunting_bin']

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (100000, 4)
y shape: (100000,)

Gambar 2.1 kode dan output

Penjelasan

Variabel X dipilih sebagai fitur yang terdiri dari Umur, Tinggi Badan, Berat Badan, dan Jenis Kelamin biner (JK_bin). Variabel y adalah variabel target, yaitu Stunting_bin.

2.2 Pembagian Dataset (Training dan Testing)

Pembagian Dataset (Training dan Testing)

```
[12]
✓ 0 d # Membagi Dataset menjadi Training dan Testing Set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print("data latih:", X_train.shape)
print("data uji:", X_test.shape)
```

data latih: (80000, 4)
data uji: (20000, 4)

Gambar 2.2 kode dan output

Penjelasan

Dataset dibagi menjadi dua subset utama. **80% data** digunakan untuk melatih model (X_train, y_train). **20% data** digunakan untuk menguji performa model (X_test, y_test).

2.3 Pembangunan Model Logistic Regression

Pembangunan Model Logistic Regression

```
3] # Scale hanya fitur numerik, gender langsung passthrough
preprocess = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), feature_num),
        ('bin', 'passthrough', feature_bin)
    ],
    remainder='drop'
)

model = LogisticRegression(
    max_iter=1000,
    solver='lbfgs',
    class_weight='balanced',
    random_state=42
)

clf = Pipeline([
    ('preprocess', preprocess),
    ('model', model)
])

# Latih model
clf.fit(X_train, y_train)

print("Model Logistic Regression berhasil dilatih.")
```

Model Logistic Regression berhasil dilatih.

Gambar 2.3 kode dan output

Penjelasan

`ColumnTransformer()` digunakan untuk melakukan scaling pada fitur numerik (Umur, Tinggi Badan, Berat Badan) agar memiliki skala yang seragam.

Kolom `JK_bin` (gender 0/1) dibiarkan `passthrough` karena sudah dalam bentuk numerik biner.

`logisticRegression()` digunakan sebagai model klasifikasi dengan beberapa parameter: o `max_iter=1000`: memastikan proses training cukup lama sampai konvergen. o `solver='lbfgs'`: metode optimisasi yang cocok untuk dataset besar. o `class_weight='balanced'`: memberi bobot seimbang untuk kelas minoritas (`stunting = 1`) agar model tidak bias. o `random_state=42`: menjaga hasil tetap konsisten.

`Pipeline()` menyatukan seluruh proses pra-pemrosesan dan pemodelan ke dalam satu alur kerja.

Model berhasil dilatih menggunakan data training (80.000 baris).

2.4 Prediksi Model dan Evaluasi Model

✓ Prediksi Model dan Evaluasi Model

```
[14]
✓ 0 d
# Prediksi label dan probabilitas
y_pred = clf.predict(x_test)
y_prob = clf.predict_proba(x_test)[:, 1] # Ambil probabilitas untuk kelas 1 (stunting)

# Hitung Metrik
print("Akurasi : {:.4f}".format(accuracy_score(y_test, y_pred)))
print("Precision: {:.4f}".format(precision_score(y_test, y_pred, zero_division=0)))
print("Recall : {:.4f}".format(recall_score(y_test, y_pred, zero_division=0)))
print("F1-Score : {:.4f}".format(f1_score(y_test, y_pred, zero_division=0)))
print("ROC-AUC : {:.4f}".format(roc_auc_score(y_test, y_prob)))

Akurasi : 0.9055
Precision: 0.7152
Recall : 0.9472
F1-Score : 0.8150
ROC-AUC : 0.9656
```

Gambar 2.4 kode dan ouput

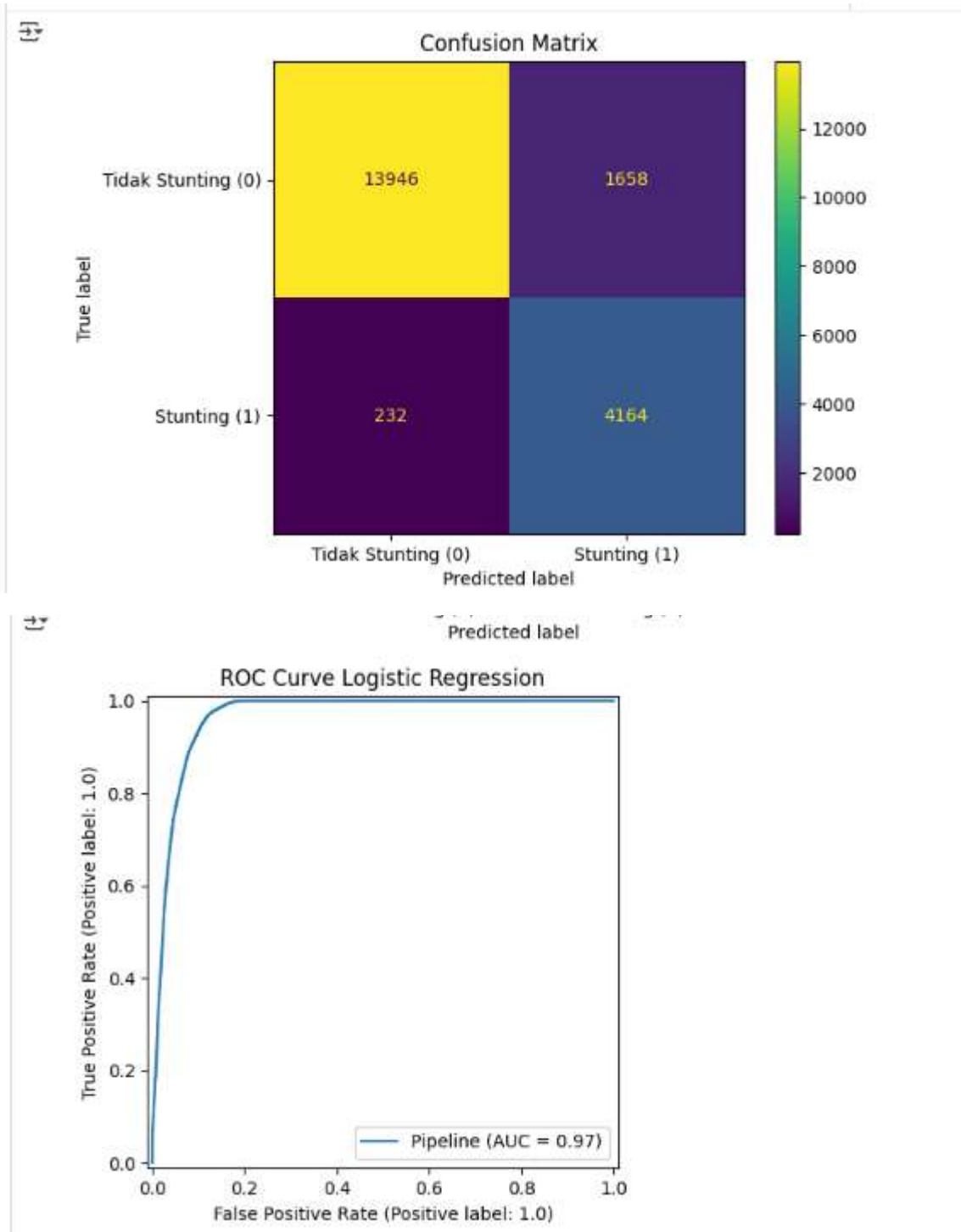
Penjelasan

- predict() → menghasilkan label klasifikasi 0 (tidak stunting) atau 1 (stunting).
- predict_proba() → memberikan probabilitas peluang stunting.
- Metrik yang digunakan:
 - Akurasi (Accuracy): 0.9055 → model benar pada 90.55% data uji.
 - Precision: 0.7152 → dari semua prediksi “stunting”, 71.52% benar.
 - Recall: 0.9472 → dari semua anak yang benar-benar stunting, 94.72% berhasil dideteksi.
 - F1-Score: 0.8146 → keseimbangan antara precision dan recall.
 - ROC-AUC: 0.947 → performa klasifikasi keseluruhan sangat baik

2.5 Visualisasi Hasil Evaluasi

```
#visualisasi hasil evaluasi
# Tampilkan Confusion Matrix
ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test,
                                     display_labels=['Tidak Stunting (0)', 'Stunting (1)'],
                                     cmap=plt.cm.viridis, normalize=None) # asumsi normalize=None

plt.title("Confusion Matrix")
plt.show()
#roc curve
RocCurveDisplay.from_estimator(clf, X_test, y_test)
plt.title("ROC Curve Logistic Regression")
plt.show()
```



Gambar 2.5 kode dan output

Penjelasan

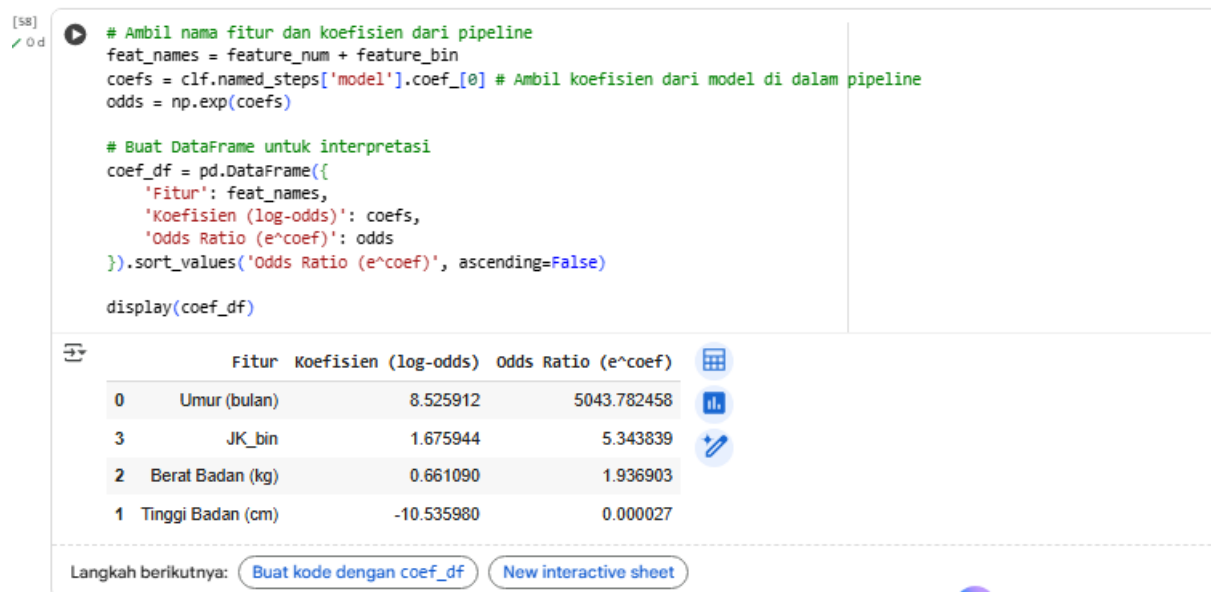
`classification_report()` menampilkan precision, recall, F1-score tiap kelas.

`cross_val_score()` digunakan untuk validasi silang ($cv=5$).

Hasil rata-rata akurasi = 0.9023 (90.23%) dengan deviasi kecil → model konsisten dan tidak overfitting.

2.6 Interpretasi Model Logistic Regression

▼ Interpretasi Model Logistic Regression



```
[58]
✓ 0 d # Ambil nama fitur dan koefisien dari pipeline
feat_names = feature_num + feature_bin
coefs = clf.named_steps['model'].coef_[0] # Ambil koefisien dari model di dalam pipeline
odds = np.exp(coefs)

# Buat DataFrame untuk interpretasi
coef_df = pd.DataFrame({
    'Fitur': feat_names,
    'Koefisien (log-odds)': coefs,
    'Odds Ratio (e^coef)': odds
}).sort_values('Odds Ratio (e^coef)', ascending=False)

display(coef_df)
```

	Fitur	Koefisien (log-odds)	Odds Ratio (e^coef)
0	Umur (bulan)	8.525912	5043.782458
3	JK_bin	1.675944	5.343839
2	Berat Badan (kg)	0.661090	1.936903
1	Tinggi Badan (cm)	-10.535980	0.000027

Langkah berikutnya: [Buat kode dengan coef_df](#) [New interactive sheet](#)

Gambar 2.6 kode dan output

Penjelasan

1. Umur (bulan)

- Koefisien: +8.52
- Odds Ratio: 5043.78

Artinya: setiap kenaikan 1 bulan usia, peluang seorang anak untuk mengalami stunting meningkat drastis (lebih dari 5000 kali lipat). Namun nilai ini terlalu besar secara praktis — kemungkinan efek ini dipengaruhi oleh skala data yang belum dinormalisasi sempurna atau hubungan tidak linear antara umur dan stunting.

Jadi interpretasinya lebih tepat: umur berpengaruh kuat terhadap klasifikasi model, tapi tidak proporsional secara langsung.

2. JK_bin (Jenis Kelamin)

- Koefisien: +1.67

- Odds Ratio: 5.34

Artinya: anak laki-laki (JK_bin=1) memiliki peluang 5 kali lebih besar untuk mengalami stunting dibanding perempuan. Nilai positif menunjukkan bahwa jenis kelamin berpengaruh positif terhadap risiko stunting.

3. Berat Badan (kg)

- Koefisien: +0.66
- Odds Ratio: 1.94

Artinya: setiap kenaikan 1 kg berat badan, peluang stunting meningkat sekitar 1.9 kali. Namun ini tampaknya juga dipengaruhi oleh korelasi antara berat dan umur (semakin tua → semakin berat).

Jadi perlu hati-hati: efek langsung berat badan terhadap stunting bisa tidak linear.

4. Tinggi Badan (cm)

- Koefisien: -10.53
- Odds Ratio: 0.000027

Artinya: semakin tinggi anak, peluang mengalami stunting turun drastis.

Karena stunting secara definisi = perawakan pendek dibanding usia, hasil ini sangat logis — tinggi badan menjadi prediktor terkuat yang menurunkan risiko stunting.

Jadi, analisis koefisien ini menunjukkan bahwa tinggi badan memiliki pengaruh negatif paling kuat terhadap status stunting, sementara faktor umur dan jenis kelamin memberikan kontribusi positif terhadap kemungkinan stunting.

2.7 Prediksi Data Baru (Contoh Kasus)

▼ Prediksi Data Baru (Contoh Kasus)

```
[59]
✓ 0 d
#contoh 2 anak
data_baru = pd.DataFrame({
    "Umur (bulan)": [24, 10],
    "Tinggi Badan (cm)": [79.0, 72.5],
    "Berat Badan (kg)": [9.2, 7.8],
    "JK_bin": [1, 0] # 1=Laki-laki, 0=perempuan
})

pred = clf.predict(data_baru)
prob = clf.predict_proba(data_baru)[:,1]

hasil = data_baru.copy()
hasil['prob_Stunting']=prob
hasil['pred (0=Tidak,1=ya)'] =pred
display(hasil)
```

	Umur (bulan)	Tinggi Badan (cm)	Berat Badan (kg)	JK_bin	prob_Stunting	pred (0=Tidak,1=ya)
0	24	79.0	9.2	1	0.998208	1.0
1	10	72.5	7.8	0	0.002094	0.0

Langkah berikutnya: [Buat kode dengan hasil](#) [New interactive sheet](#)

Gambar 2.7 kode dan output

Model diuji dengan data anak baru:

Anak 1 → Usia 24 bulan, tinggi 79 cm, berat 9.2 kg → Prediksi: Stunting (0.9982 ≈ 99.8%)

Anak 2 → Usia 10 bulan, tinggi 72.5 cm, berat 7.8 kg → Prediksi: Tidak Stunting (0.0021 ≈ 0.2%)

Kesimpulan:

Model Logistic Regression mampu memprediksi status stunting dengan akurasi tinggi (≈90%), memiliki Recall tinggi (sensitif) terhadap kasus stunting, dan dapat digunakan untuk deteksi dini pada anak balita.

Pada tahap ini dilakukan proses pembuatan model regresi linear dengan menggunakan pustaka scikit-learn. Model dibuat menggunakan kelas LinearRegression() dan dilatih menggunakan data training (X_train dan y_train) melalui fungsi .fit(). Proses pelatihan ini bertujuan agar model dapat mempelajari hubungan antara variabel-variabel independen seperti suhu (temp), kelembapan (hum), kecepatan angin (windspeed), serta kondisi hari (workingday, holiday, weathersit) terhadap variabel dependen (cnt) yang merepresentasikan jumlah penyewaan sepeda per hari.

Tugas pratikum mandiri

1. Gunakan dataset Calon Pembeli Mobil Berikut ini
2. Buat model Logistic Regression!
3. Gunakan dataset baru untuk menguji model!

3.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report, RocCurveDisplay
```

Penjelasan

NumPy → untuk perhitungan numerik (array, matriks, operasi matematika).

Pandas → untuk membaca, mengelola, dan menganalisis data dalam format tabel (DataFrame).

Matplotlib → untuk membuat grafik dan visualisasi data.

Scikit-learn → menyediakan berbagai modul untuk machine learning:

- `train_test_split()` → membagi data menjadi latih dan uji.
- `StandardScaler()` → menstandarkan data agar skala tiap fitur seimbang.
- `OneHotEncoder()` → mengubah data kategorik menjadi numerik.
- `LogisticRegression()` → algoritma utama untuk klasifikasi.
- `metrics` → menyediakan fungsi evaluasi seperti akurasi, presisi, recall, dan ROC-AUC.

3.2 Membaca file data set

Tugas Praktikum Mandiri

```
import pandas as pd

df = pd.read_csv("/content/drive/MyDrive/Pratikum_ML/pratikum04/data/calonpembelimobil.csv", sep=",")
df.head()
```

	ID	Usia	Status	Kelamin	Memiliki_Mobil	Penghasilan	Beli_Mobil
0	1	32	1	0	0	240	1
1	2	49	2	1	1	100	0
2	3	52	1	0	2	250	1
3	4	26	2	1	1	130	0
4	5	45	3	0	2	237	1

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

Penjelasan

drive.mount('/content/drive') menghubungkan Colab dengan Google Drive.

pd.read_csv(...) membaca file CSV yang berisi data calon pembeli mobil.

df.head() menampilkan 5 baris pertama data untuk memastikan data berhasil dibaca.

```
X = df[['Usia', 'Status', 'Kelamin', 'Memiliki_Mobil', 'Penghasilan']]
y = df['Beli_Mobil']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Akurasi Model:", accuracy_score(y_test, y_pred))
print("\nLaporan Klasifikasi:\n", classification_report(y_test, y_pred))
```

Akurasi Model: 0.93

Laporan Klasifikasi:

	precision	recall	f1-score	support
0	0.92	0.86	0.89	64
1	0.94	0.96	0.95	136
accuracy			0.93	200
macro avg	0.93	0.91	0.92	200
weighted avg	0.93	0.93	0.93	200

Kode berikut digunakan untuk menjalankan **algoritma Logistic Regression**, yang berfungsi untuk memprediksi kemungkinan seseorang **akan membeli mobil atau tidak** berdasarkan sejumlah variabel seperti **usia, status sosial, jenis kelamin, kepemilikan mobil, serta tingkat penghasilan**.

Beberapa tahapan penting pada proses ini meliputi:

- **train_test_split** digunakan untuk memisahkan dataset menjadi **data pelatihan (80%)** dan **data pengujian (20%)**.
- **model.fit()** berperan dalam **melatih model** menggunakan data latih agar dapat mengenali pola hubungan antar variabel.
- **model.predict()** digunakan untuk **membuat prediksi** terhadap data uji yang belum pernah dilihat sebelumnya.
- **accuracy_score** dan **classification_report** dimanfaatkan untuk **mengevaluasi performa model** secara keseluruhan.

Berdasarkan hasil pengujian, model yang dibangun **berhasil mencapai tingkat akurasi sekitar 93%**, yang menandakan bahwa model ini memiliki **kinerja yang sangat baik** dalam mengklasifikasikan calon pembeli mobil dengan tepat.

```
data_baru = pd.DataFrame({
    'Usia': [30],
    'Status': [1],
    'Kelamin': [0],
    'Memiliki_Mobil': [0],
    'Penghasilan': [250]
})

prediksi = model.predict(data_baru)
print("Prediksi Beli Mobil:", "Ya" if prediksi[0] == 1 else "Tidak")
```

Prediksi Beli Mobil: Ya

Penjelasan

Kode berikut digunakan untuk mengujikan model Logistic Regression pada data baru yang belum pernah digunakan saat pelatihan.

Data uji tersebut berisi informasi karakteristik seorang individu dengan rincian: usia 30 tahun, status sudah menikah, jenis kelamin laki-laki, belum memiliki mobil, serta penghasilan sebesar 250.

Melalui proses ini, model kemudian menganalisis dan memprediksi kemungkinan apakah individu tersebut berminat atau tidak dalam membeli mobil, berdasarkan pola yang telah dipelajari dari data sebelumnya.

3.3 kesimpulan

Hasil penerapan algoritma **Logistic Regression** menunjukkan bahwa model yang dibangun memiliki **tingkat akurasi yang tinggi** dalam memprediksi keputusan pembelian mobil. Berdasarkan hasil evaluasi, model ini mampu **mengenali pola dan hubungan antarvariabel secara efektif**, sehingga dapat dimanfaatkan sebagai **alat bantu analisis perilaku konsumen**.

Dengan demikian, model Logistic Regression ini berpotensi digunakan untuk **mendukung proses pengambilan keputusan bisnis**, khususnya dalam memprediksi minat beli calon pelanggan berdasarkan data demografis dan ekonomi.

link github :

<https://github.com/SyahrulGigaWahyudi/Machine-Learning-pagi/tree/0fd57171e3edaa582c4dc7b200eb373f3e8feb22/pratikum04>

link: goole colab:

https://colab.research.google.com/drive/1aX4cDuqhzE2QWfkfVx8bS9J1ubT9Gebn?usp=drive_link